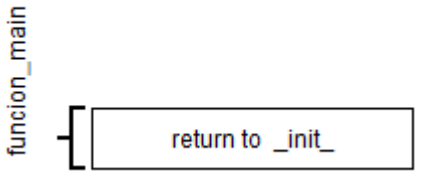
Conceptos Avanzados de Lenguajes de Programación

Trabajo Practico Obligatorio 2

Alumno: Iovaldi, Sebastián Ángel
Legajo: fai-3482

Ejercicio 1: Dado el siguiente Programa...

Static data			
g[3]	m	n	p
{na, na, na}	na	na	na

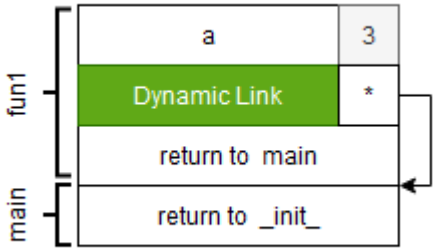


La función main no tiene variables locales que alocar en el stack

```
int main ()
{
    m=3; n=1;
    g[0]=5; g[1]=4; g[2]=2;
```

g[3]	m	n	p
{5, 4, 2}	3	1	na

```
    try
    { // externo
        printf( "m, n, g[0] = " , m, n, g[0]); /* out 1 */
        try
        { // interno
    --->    printf(m , fun1(m), g[p], g[m]);
        }
```

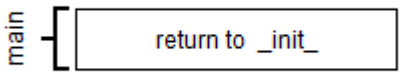


```
int fun1 ( int a )
{
    try
    {
  --->  a = g[a];   /* fuera de rango */
        printf ( "fun1 g(a,n,p) = " , g[a] , g[n] , g[p]) ;
    }
```

```
    catch (ExcepcionIndiceFueraDeRango)
    {
        a = (-1 * a ) + 4;      /* a = (-1*3)+4 = 1 */
        p = 1;
        printf ( a , p ) ;      /* out 2* /
    }
```

g[3]	m	n	p
{5, 4, 2}	3	1	1

```
    finally
    {
        printf ( "en fun1: 2a, 2p=" , a+a,p*2 );  /* out 3 */
    }
    return (a+1);
}
```




```
        try
        { // interno
   --->     printf(m , fun1(m), g[p], g[m]);   /* fuera de rango */
        }
```

```
        catch (ExcepcionIndiceFueraDeRango)
        {
  --->      p = g[0] / g [m];         /* fuera de rango */
        }
```

```
    catch (ExcepcionIndiceFueraDeRango)
    {
        m = m - 1;
```

g[3]	m	n	p
{5, 4, 2}	2	1	1

```
        printf ( "g[m] = ", g[m]);    /* out 4 */
        p = g[0] / g[m]; /* p = 5/2 = 2 */
    }
```

g[3]	m	n	p
{5, 4, 2}	2	1	2

```
    finally
    {
        printf( "n , g(m) en finally externo=", n , g[m]) ;  /* out 5*/
    }

    m = n - p;
    printf ( "m, n, p =" , m, n, p);  /*out 6*/
}
```

g[3]	m	n	p
{5, 4, 2}	-1	1	2

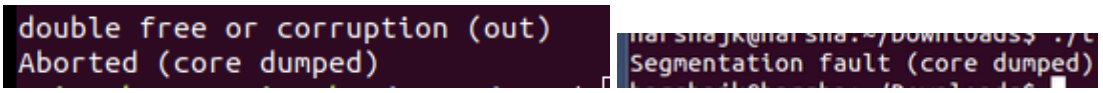
Output

```
1    m, n, g[0] = 3, 1, 5
2    1, 1
3    en fun1: 2a, 2p= 2, 2
4    g[m] = 2
5    n, g(m) en finally externo= 1, 2
6    m, n, p = -1, 1, 2
```

Ejercicio 2: Investigar y Describir cómo se administra la memoria Heap en el lenguaje RUST. Denir y explicar el concepto de Propiedad (Ownership) y de que manera soluciona las referencias sueltas y garbage. Ejemplique.

La memoria declarada en el stack tiene tamaño fijo, se aloca entrando en el scope de un bloque o función y se libera al salir. Puede llamarsele memoria *automática*.

Cuando la cantidad de memoria requerida en un programa no es constante ni conocida en tiempo de compilación, puede reservarse memoria en el Heap.
La memoria del Heap debe ser gestionada mediante *system calls* al sistema operativo, por lo que es más lenta de gestionar y debe ser reservada y liberada explicitamente.
La gestión de este segmento de memoria puede inducir errores de programación que no son detectados en tiempo de compilación...



Rust fuerza a los desarrolladores a utilizar la memoria Heap a través de una tipo de dato *Box* declarado en el Stack y que encapsula la memoria Heap.
Así, cuando *Box* se libera de forma automática al salir de *scope*, llama a una función *drop*, que libera su contenido reservado en el Heap. De esta manera no es necesario que el desarrollador gestione explicitamente la memoria dinámica.

El concepto es análogo y *borrowed*, del de RAII de C++ (*resource acquisition is initialization*). Los objetos inicializan y liberan los recursos de los que son dueños en sus constructores/destructores (smartpointers).

```
{
    let s = String::from("hola");     /*s es un objeto en el stack
                                      String reserva espacio en el Heap
                                       cuando es construído*/
}
  /*cuando s sale de scope, string llama a drop y libera sus recursos*/
```

Pero, ¿Quién es el dueño de los recursos? Ownsership...

Rust fuerza al desarrollador a seguir su propio de modelo de *ownsership* que sigue tres reglas:

- Cada valor tiene *owner*
- Solo puede haber un *owner* a la vez
- Cuando el *owner* sale de *scope* el valor es descartado.

En C++ este modelo es solo un *idiom.*

La ventaja de forzar el modelo, es que el compilador de Rust puede, con ayuda del *Borrow Checker*, realizar un análisis estático que previene errores en la gestión de memoria de los programas.

El borrow checker limita la cantidad de referencias (punteros no nullables) de los objetos, garantiza que no se acceda a *dangling pointers*, es decir punteros que no son válidos por una liberación de memoria previa y previene que los datos se *muevan* de tal forma que pongan en riesgo el *Ownership*.

Rust...

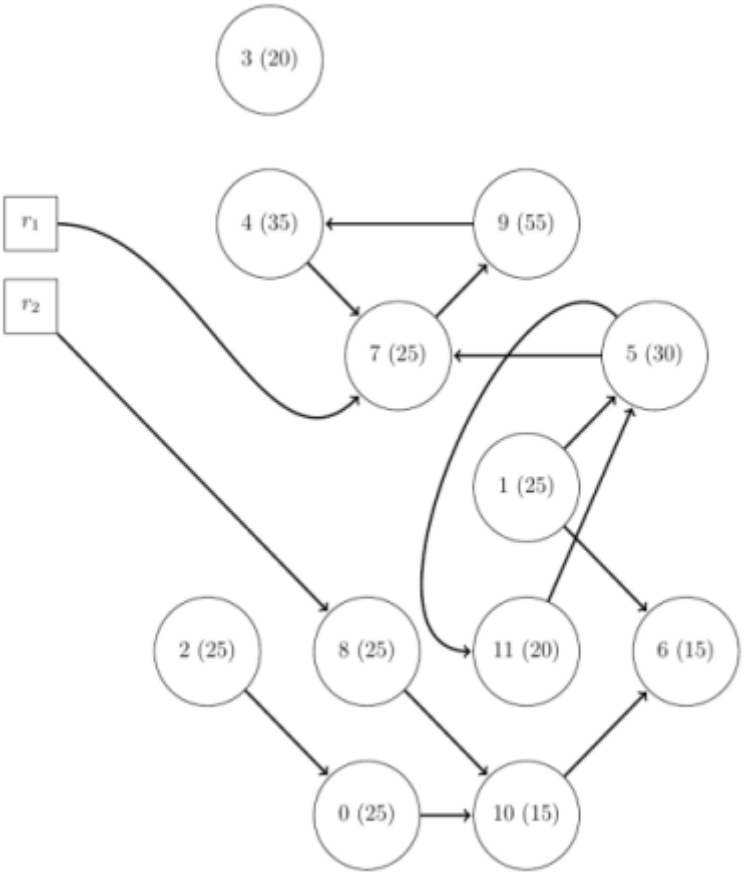
```rust
/* retorna un referencia al objeto string, declarado en el stack*/
fn crear_string() -> &String
{
    let s = String::from("hello");
    &s/* error del compilador! s deja de existir al terminar fn*/
}
```

C++...

```cpp
std::string& crear_string()
{
    std::string s("hello");
    return s;
}

/* Este código hace lo mismo y compila perfectamente! */
/* con Undefined Behavior... */
```

Ejercicio 3: Dado el siguiente Esquema de Memoria Heap donde el numero es el orden en el heap y entre parentesis el tamaño del bloque:



Dar la denición de los siguientes algoritmos y ejemplicar con este esquema de memoria.

1. Tombstones y locks ands keys.
2. Gestión de memoria dinámica y garbage collector:
    a. Algortimo de conteo de referencias
    b. Algoritmo de Mark and Sweep
    c. Algoritmo de marcado tricolor.

1.

Tombstones y Locks-And-Keys son dos algoritmos que proveen una solución a los *dangling pointers* (punteros colagentes)

Una **tombstone** es una estructura que actúa como intermediaria entre un puntero y su objecto, usualmente datos del Heap. El *handler*, apunta únicamente a las tombstones y nunca directamente a su objeto real y lleva una variable que indica el estado actual del objeto reservado. Cuando los datos son liberados, la tombstone cambia su valor, indicando que la variable ya no existe.

La clase std::weak_ptr de la libraria estandar de C++ utiliza este patrón.

En **Locks-And-Keys** se representan los punteros como pares ordenados (clave, dirección), donde la clave es un valor entero. Las variables Heap se representan como el tamaño de almacenamiento más una celda para un valor entero llamado Lock.

Cuando una variable es asignada, se crea un valor de Lock que se coloca tanto en la celda de la variable como en la clave del puntero. Cada acceso al puntero compara estos dos valores, y el acceso solo se permite si los valores coinciden.

Cuando una variable es liberada, la clave del puntero se modifica para contener un valor diferente al de la celda de la variable. A partir de ese momento, cualquier intento de desreferenciar el puntero puede marcarse como un error. Dado que copiar un puntero también copia su valor de celda, cambiar la clave del par ordenado deshabilita de manera segura todas las copias del puntero.

2.

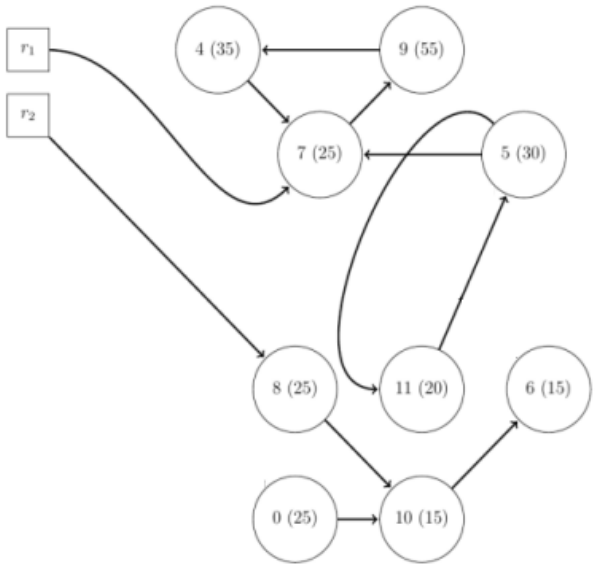
Conteo de Referencias

Cada objeto que quiere referenciarse lleva un contador. Cuando se crea una referencia a dicha objeto, se incrementa el contador en uno. Cuando se pierde una referencia se descuenta uno.

Cuando el objeto ya no tiene referencias que le apunten, el objeto queda inaccesible, lo que generaría un *memory leak*. El truco es que cuando esto sucede, el objeto se libera de memoria.

En el esquema de memoria, se ve como por ejemplo, el objeto 6 tiene 2 contadores.

Los objetos 2, 3 y 1 tiene cero, por lo tanto son los proximos a ser liberados



Luego el contador del objeto 0, llega a cero y también es liberado.

Mark and Sweep

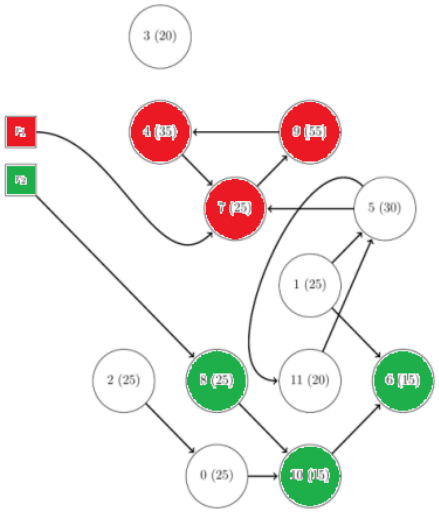
El recolector de basura gestiona una arbol de referencias, donde desde la raíz se pueden obtener mediante punteros todos los objetos que fueron reservados.

Cada nodo del arbol representa un objeto y un estado de validez, que comienza en inválido.

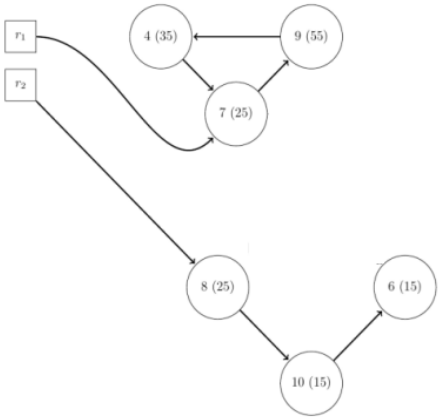
El algoritmo consiste en recorrer todos los objetos accesibles, marcandolos como válidos. Luego se liberan los nodos no accesibles.

El algoritmo no puede interrumpirse a mitad de la ejecución y es problemático por su larga duración.

En el ejemplo, el GC recorre desde las raíces en orden: r1 → 7 → 9 → 4 y luego r2 → 8 → 10 → 6 y los marca como válidos (los nodos de colores son válidos y los blancos inválidos, solo es con fin visual, solo hay estados posibles).



Luego recorre los nodos que siguen marcados como inválidos y los libera.



Marcado Tricolor

Es una mejora del Mark and Sweep, que le brinda la posibilidad de reducir el trabajo en partes.

Se basa en clasificar los objetos en tres conjuntos o "colores":

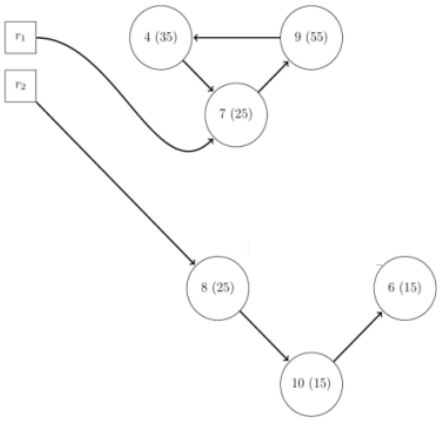
1. Blanco: Objetos aún no examinados y que podrían ser basura.
2. Gris: Objetos accesibles pero cuyos hijos aún no han sido examinados.
3. Negro: Objetos accesibles cuyos hijos ya han sido examinados.

El GC (recollector de basura) comienza marcando los objetos raíz como grises.

Procesa los objetos grises, pintandolos de negro después de examinar sus hijos y volviendolos grises.

Cuando ya no hay objetos grises, todos los objetos blancos son considerados inaccesibles y se eliminan.

El resultado es el mismo que con Mark and Sweep, pero la ejecución puede hacerse en paralelo y puede ser pausada y reanudada.



Ejercicio 4: Use su lenguaje favorito (el que mas tiempo use que tenga capacidad para escribir scripts) para crear un script que genere una version pequeña (thumbnail) de todas las imagenes en formato jpg en un directorio.

Dependencias: lua5.1, magick

```lua
local magick = require("magick")
local lfs = require("lfs")

local function generar_thumbnails(directorio, tamano_thumbnail)
    if not lfs.attributes(directorio, "mode") then
        print("El directorio especificado no existe: " .. directorio)
        return
    end

    local directorio_salida = directorio .. "/thumbnails"
    lfs.mkdir(directorio_salida)

    for archivo in lfs.dir(directorio) do
        if archivo:match("%.jpg$") then
            local ruta_entrada = directorio .. "/" .. archivo
            local ruta_salida = directorio_salida .. "/" .. archivo

            local status, err = pcall(function()
                local img = magick.load_image(ruta_entrada)
                img:resize(tamano_thumbnail, tamano_thumbnail)
                img:write(ruta_salida)
                print("Miniatura creada: " .. ruta_salida)
            end)

            if not status then
                print("Error procesando '" .. archivo .. "': " .. err)
            end
        end
    end
end

io.write("Ingrese la ruta del directorio con las imágenes JPG: ")
local directorio = io.read("*l")
local tamano_thumbnail = 128
generar_thumbnails(directorio, tamano_thumbnail)
```

```
seba@desktop:~/fai/calp/tpo02$ lua5.1 thumbails.lua
Ingrese la ruta del directorio con las imágenes JPG: images
Miniatura creada: images/thumbnails/gato.jpg
Miniatura creada: images/thumbnails/leon.jpg
Miniatura creada: images/thumbnails/husky.jpg
Miniatura creada: images/thumbnails/loto.jpg
Miniatura creada: images/thumbnails/tigre.jpg
Miniatura creada: images/thumbnails/aguila.jpg
seba@desktop:~/fai/calp/tpo02$
```

